Prompting for Operators

Instructor guide · Day 1 · Session 2

Roles, context, structured outputs, reusable prompt templates, and AI assistants with projects and files

Session at a glance

Day / session	Day 1 · Session 2 · 120 minutes
Lead instructors	Robina Mirbahar
Online module	https://albanna-tutorials.com/ai-administrator/02_prompting_for_operators.html
Deck	Day1_S2_Prompting_for_Operators.pptx, 53 slides, speaker notes included

Learning objectives

- Use the five parts of a working prompt: role, context, task, constraints, output format.
- Ask for structured outputs (tables, checklists, labelled fields) that can be copied into a spreadsheet or a workflow.
- Turn a good prompt into a reusable template with placeholders, and improve it with few-shot examples and a ten-row test set.
- Set up a project or custom assistant with uploaded documents, and test where it is reliable and where it is not.
- Recognise hallucination and over-confidence, and apply the three habits that catch them.

Run of show

Minutes	Block	Focus
2 to 10	1. A prompt is a work instruction	Vague instruction, vague output
10 to 35	2. The five parts, with a before and after	A real request from an office coordinator
35 to 55	3. Structured outputs and few-shot examples	An automation cannot read prose. It reads fields.
65 to 85	4. Assistants that know your documents	Projects, custom GPTs, Gems, Copilot agents
85 to 95	5. Where prompts fail: hallucination and over-confidence	A plausible fact looks exactly like a correct one

95 to 112	Practical lab: prompt library and department assistant	Steps shown now, finish at home, due before the next session
112 to 120	Kahoot: knowledge check	20 questions, 20 seconds each. Phones out.

The welcome, objectives and run-of-show slides take the first minutes before Section 1. The break is planned at the midpoint (see the run-of-show slide in the deck). Cuts if running behind are listed under Troubleshooting.

Materials checklist

- Projector or screen share; this deck (Day1_S2_Prompting_for_Operators.pptx) with speaker notes open.
- Two assistants logged in for the demo, ideally ChatGPT (Projects or a custom GPT) and Claude (Projects); Copilot or Gemini Gems as a third if available.
- Two public policy documents as PDFs (for example a university leave policy and a travel expenses policy found online) ready to upload in the demo and to share with participants who have none.
- One sample maintenance-request email (reuse the Session 1 demo email).
- Lab handout (Day1_S2_Lab_Handout.pdf) and a spreadsheet template with three columns (Input, Expected, Model answer) shared at minute 95.
- Kahoot imported from Day1_S2_Kahoot.xlsx and ready to launch (PIN on screen at minute 112); answer key Day1_S2_Kahoot_Answer_Key.pdf. The written Quiz bank (Day1_S2_Quiz_Bank.pdf) stays available for homework.
- Both course bots running on the course server (hermes gateway status as user hermes): @aicaravan_bot as prompt test bench, @aicaravan_policy_bot as the grounded policy assistant. See Day1_S1_Hermes_Setup_Guide.pdf, Session 2 section.

Before the session

- Read the live module once (link above); the slides follow its five sections in order.
- Build the department assistant yourself before the session (slide 'Assistant instructions') and run the trap test, so you know what it does with your two documents.
- Run the few-shot classifier on five made-up messages so the demo has no surprises.
- Check the data terms of the consumer tier you will demo on; say out loud that only public documents are used.

Live demo: from a vague request to a template to an assistant (minutes 10 to 85, in three short bursts)

1. Burst 1 (minute 15): type the 'Before' prompt with the sample email, show the generic answer. Then paste the 'After' prompt. Show the five labelled lines. Change the email slightly so urgency is ambiguous; show that the model writes 'unclear'.
2. Burst 2 (minute 45): in Telegram, paste the few-shot classifier plus five test inputs with expected labels into @aicaravan_bot; it returns the results table and a score. Change one example, paste again, show the score move. This is what students do with their ten rows in the lab.

3. Burst 3 (minute 70): create a Project (ChatGPT or Claude), paste the assistant instructions, upload two policies, ask one covered and one trap question. Then ask the same two questions to @aicaravan_policy_bot and, with the server log on screen (journalctl --user -u hermes-gateway -f as user hermes), point out that the question and the policy text travel to the model provider: data goes where the model goes.
4. If time allows, show the same Project idea in Copilot or Gemini Gems in one minute each, to make the point that the feature names differ but the idea is identical.

Slide-by-slide talking points

Slide numbers match the deck. Quiz questions occupy two slides each (question, then answer).

1. Prompting for Operators · **Title**
Welcome. Recap Session 1 in two sentences: an agent is model, tools, memory and instructions; the instructions are the part you own. This session is about writing those instructions so they work inside an automation, not just in a chat.

2. What you will be able to do after this session · **Bullets**
Read quickly. Mention the Kahoot so phones are ready. The deliverable is a five-template prompt library plus a tested department assistant. Both are reused on Day 3 (the templates go into n8n AI nodes) and Day 4 (the assistant becomes an agent).

3. Run of show (120 minutes) · **Table**
The demo is split into three short bursts so participants see each idea working right after hearing it. Keep each burst under five minutes.

4. 1. A prompt is a work instruction · **Section** · **minute 2**

5. How you brief a new temp · **Bullets**
The temp analogy carries the whole session. Every time a prompt fails, ask 'what would a temp have needed to know?'. Participants who manage people find this immediately natural.

6. The standard: repeatability · **Bullets**
This reframes prompting from a creative skill to an operations skill. The test is not 'did I like the answer' but 'would the same prompt give the same shape of answer on the next hundred emails'.

7. Why the model needs everything spelled out · **Figure** · **figure: context-window-memory**
Link to Session 1: the context window is the model's whole world for this task. Your policies, your definitions of 'urgent', your team's vocabulary are not in it unless you put them there. That is what the 'context' part of a prompt does.

8. The five parts of a working prompt · **Steps**
Most weak prompts are missing two of these, usually context and output format. Have participants hold up fingers for the five parts; you will refer to them by number for the rest of the session.

9. 2. The five parts, with a before and after · **Section** · **minute 10**

10. Before · **Prompt text**
Run it live (demo burst 1). The answer will be fluent and useless: a paragraph, a different one each time, and the model has decided for itself what 'what to do' means.

11. After · **Prompt text**
Run it on the same email. Point at each of the five parts as you read. Then edit the email so the urgency is genuinely ambiguous and run again: the model writes 'unclear'. That moment is the whole point of constraints.

12. What each part does, and what goes wrong without it · **Table**
The right-hand column is a diagnostic table. When a prompt misbehaves, find the symptom here and add the missing part. Participants will use the same idea in the symptom table at the end of the session.

13. The most valuable sentence in the prompt · **Callout**
Say it twice. On Day 3 the 'unclear' output becomes a branch in n8n that routes the item to a person. On Day 5 it is the basis of the human-in-the-loop pattern. One sentence in a prompt, one control in a workflow.

14. Role is powerful enough to build whole systems on · **Figure** · **figure: camel-roles**
Thirty seconds. The point for operators: the 'Role' line is not decoration. It changes vocabulary, assumed knowledge and tone so much that research systems are built from nothing but roles talking to each other. Skip if behind.

15. Roles in collaboration · **Figure** · **figure: camel-collaboration**
Optional. Preview of Day 5's multi-agent patterns. Keep it to one sentence and move on.

16. 3. Structured outputs and few-shot examples · **Section** · **minute 35**

17. Ask for fields, and say the allowed values out loud · **Bullets**
Show the triage note from the 'After' prompt again and point out that each labelled line is a column. On Day 3 those lines are mapped to spreadsheet columns and Slack messages with no code.

18. Few-shot examples: show the pattern instead of describing it · **Prompt text**
Demo burst 2. Classify three messages, one awkward ('My invoice is wrong AND the portal is down'). Then change an example and show the classification move. Models copy patterns better than they follow descriptions; two or three worked examples beat a paragraph of rules.

19. Examples with reasoning: chain-of-thought · **Figure** · **figure: chain-of-thought-prompting**
For operators: if a classification needs a judgement ('is this a complaint or a question?'), write the example as 'message -> reason -> category'. The model will then give its reason, which a reviewer can check.

20. The one-line version: 'think step by step' · **Figure** · **figure: zero-shot-chain-of-thought**
Cheap trick worth knowing. Add 'Work through the steps before giving the final answer' to the constraints when the task has more than one step. Not needed for simple classification; useful for summaries that must reconcile several sources.

21. Ask for the reasoning, then the answer · **Figure** · **figure: reasoning-process**
Link to Day 5 and Day 6: a model that writes 'Urgent because the email mentions a stopped lift' is auditable; one that writes only 'Urgent' is not. Put the reason line in the output format when a human will review.

22. One verb, one deliverable: break big tasks into steps · **Figure** · **figure: planning-process**
'Summarise this and tell me what to do' is two tasks. Make it two prompts, or one prompt with two clearly labelled output sections. On Day 3 the split becomes two AI nodes in a row.

23. Iterating on a prompt · **Steps**
Open the three-column sheet in the demo and do two rows. The discipline is re-running all ten, not just the ones that failed; fixes often break something that worked.

24. Models can refine their own output (Self-Refine) · **Figure** · **figure: self-refine-process**
Useful for drafting tasks (replies, reports). Ask the model to review its own draft against the constraints before output. Costs more tokens, catches obvious misses. It does not replace your ten-row test set.

25. Learning from mistakes (Reflexion) · Figure · figure: reflexion-process
Two sentences: research agents keep a memory of failures to improve the next run. Your ten-row sheet and the notes you add to a template are the same loop, run by you. Skip if behind.

26. That ten-row sheet is your first test set · Callout
Participants tend to throw the sheet away once the prompt works. Make the case for keeping it: models get updated by vendors without notice; the sheet is how you find out.

27. Try it on the live agent · Join (QR)
The Day 1 bot now acts as a prompt test bench. Demo burst 2 can use it instead of the three-column sheet: paste the few-shot classifier and five inputs, show the table and the score, change one example, paste again. Students do the same in the lab with their ten rows.

28. 4. Assistants that know your documents · Section · minute 65

29. The names differ; the idea is the same · Table
Every major assistant lets you save standing instructions together with uploaded files so you do not repeat the context each time. Ask who uses which product at work; most rooms are split between Microsoft and Google, with ChatGPT on personal accounts.

30. How documents become memory: summarise what does not fit · Figure · figure: memory-summarization
First of two mechanisms. When a document or a long conversation would not fit, the platform summarises it. Good for gist, bad for exact clauses, which is why the next mechanism exists.

31. How documents become memory: index them · Figure · figure: vector-database-creation
Keep it to the metaphor of a very good index at the back of a book. No maths. This is what happens when you click 'upload' in a Project. Day 4 calls this RAG and builds it in n8n.

32. ...and fetch only the relevant passage · Figure · figure: vector-database-retrieval
This explains both the power and the failure mode: if the right passage is not retrieved, the model answers from general knowledge and may invent. Hence the instruction 'quote the sentence you relied on'.

33. The department assistant: the simplest agent you will meet · Bullets
Tie back to the four parts from Session 1: three of four present. Participants build exactly this in the lab today and extend it on Day 4.

34. Preview: memory at full scale (Generative Agents, 2023) · Figure · figure: generative-agents
Optional, one sentence. Shows where document memory leads: agents that remember, reflect and plan over long periods. Day 5 touches on this with always-on personal agents.

35. Data goes where the model goes · Callout
Non-negotiable. State clearly that today's lab uses public documents only. If anyone has uploaded a real internal file during the session, ask them to delete the Project afterwards.

36. 5. Where prompts fail: hallucination and over-confidence · Section · minute 85

37. What hallucination is · Figure · figure: llm-memory-issues
Return to the first idea of the day: next-word prediction. Hallucination is not a bug to be patched away; it is what the mechanism does when starved of facts. The remedy is to feed it facts and give it an exit.

38. Three habits that catch most of it · Steps
These three go on the wall. They also map onto the course: grounding is Day 4 (RAG), the exit is Day 5 (routing to a human), checking the checkable is Day 6 (evaluation).

39. Symptom, likely cause, fix · Table
Participants can use this table during the lab when the classifier misbehaves. Every fix is a missing prompt part or a missing example; none of them is 'be more accurate'.

40. Tools change fast; your templates and test set travel with you · Figure · figure: framework-growth
Closing thought before the lab. Participants worry about picking the wrong product. The skills in this session are product-independent: a template written in five parts pastes into ChatGPT, Claude, Copilot, Gemini or an n8n node unchanged.

41. Practical lab: prompt library and department assistant · Section · minute 95

42. Lab steps (1 to 4) · Steps
Share the handout and the three-column sheet. Most participants will write two or three templates in the room and finish the rest at home. Make sure everyone reaches step 4 at least once so the test-set habit is formed.

43. Lab steps (5 to 7) · Steps
The trap test is the most memorable five minutes of the day. Ask two or three participants to report what their assistant did with the trap questions.

44. Trap test: two assistants, same five questions · Two columns
The point: two assistants, same instructions, same trap questions. The course bot is the reference run; the instructor can show the gateway log while a student asks it something, to make "data goes where the model goes" concrete.

45. Template skeleton (step 2) · Prompt text
Point out that the constraints line is pre-filled with the unsure clause. Participants should never remove it.

46. Assistant instructions (step 6) · Prompt text
Demo burst 3 uses exactly this text. Note the three habits inside it: grounded (only from the documents), an exit (not covered), and checkable (quote the sentence).

47. Deliverable · Callout
The templates are reused on Day 3 inside n8n AI nodes, so quality here saves time later.

48. Kahoot: knowledge check · Section · minute 112

49. Join the Kahoot · Two columns
Import Day1_S2_Kahoot.xlsx into Kahoot beforehand (Create, then Import spreadsheet). Launch in Classic mode and show the PIN on this slide. Eight minutes covers roughly 15 questions; assign the remainder as a challenge. The answer key with rationales is Day1_S2_Kahoot_Answer_Key.pdf.

50. Key takeaways · Bullets
Read the five takeaways. Then set up the next session (Day 2, workflow mapping): bring the M1 task inventory and the M2 prompt library; next time we map the chosen task as a current-state process in Miro or draw.io.

51. Preview of Day 5: many assistants, one system · Figure · figure: multi-agent-components
Close on the big picture. Day 2 goes back to the process itself: before building anything, map it.

52. Latest reading on prompting for work (2026) · Latest articles
Optional two minutes, or leave for self-study. Each item ties a recent piece to something taught today. Instructors may swap in newer pieces; keep the pattern: source, date, one-line takeaway tied to the session.

53. References · References
Leave on screen during questions. All diagrams in this deck are from Alammar and Grootendorst, Hands-On Large Language Models (O'Reilly, 2024), credited on each slide; the same figures appear in Grootendorst's online Visual Guide.

Troubleshooting and cuts

If	Then
Participants have no documents to upload	Share your two public policy PDFs in the chat. Any public policy works; the content matters less than the trap test.
Consumer-tier assistant will not accept uploads or has no Projects feature	Paste the policy text directly into the conversation as context. The 'assistants that know your documents' idea still holds; note that pasting is the context-window version, uploading is the long-term-memory version.
The classifier scores 10/10 on the first try	Ask the participant to add two more awkward inputs (mixed intent, sarcasm, a message in another language) until something fails. The lab is about the fix-and-re-run habit, not the score.
Running behind at the break	Cut the CAMEL and Generative Agents slides (role-prompting and memory previews) and shorten the products table to one minute. Do not cut the trap test in the lab.

Latest articles used on the slides

- **Prompting best practices (official Claude platform documentation)**. Anthropic, living document updated with each model release, 2026. platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
  Be explicit about output format and constraints, give 2 to 5 worked examples, and separate context, instructions and examples with clear labels. The vendor's own version of today's five parts and few-shot examples.
- **Prompt engineering for business: the 2026 practical guide**. Gend, 5 January 2026 (updated 30 March 2026). gend.co/blog/prompt-engineering-for-business
  Proposes a business template, R-TCC-COE: role, task, context, content, constraints, output, evaluation. Same skeleton as today's template, plus an explicit evaluation rubric, which is what your ten-row test set provides.
- **Production prompt engineering in 2026: structured outputs, prompt chaining and optimisation**. AI Workflow Lab, by Marcus Holloway, 13 February 2026 (updated 2 June 2026). aiworkflowlab.dev/article/production-prompt-engineering-2026-structured-outputs-prompt-chaining-dspy
  Treat prompts as engineered, versioned, tested artefacts; constrain outputs to a fixed shape; split big tasks into chains of small prompts. The 'one verb, one deliverable' and 'fields not prose' rules, written for engineers.
- **Context Engineering: from prompts to corporate multi-agent architecture**. V. Vishnyakova, arXiv 2603.09619, 10 March 2026. arxiv.org/abs/2603.09619
  Beyond the prompt, what matters is the whole information environment the model works in: documents, definitions, memory. Today's 'context' part and the department assistant's uploaded policies are the first step of that discipline.

References

1. Alammar, J. and Grootendorst, M. (2024). Hands-On Large Language Models. O'Reilly Media (source of the figures in this deck)

2. Al-Banna, A. (2026). AI Administrator, Module 2: Prompting for Operators. albanna-tutorials.com/ai-administrator/02_prompting_for_operators.html
3. Al-Banna, A. (2025). LLM Agents, explained visually (tutorial). albanna-tutorials.com/llm_agents.html
4. Grootendorst, M. (2025). A Visual Guide to LLM Agents. newsletter.maartengrootendorst.com (online version of the same diagrams)
5. Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022
6. Kojima, T. et al. (2022). Large Language Models are Zero-Shot Reasoners. NeurIPS 2022
7. Madaan, A. et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023
8. Shinn, N. et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023
9. Li, G. et al. (2023). CAMEL: Communicative Agents for 'Mind' Exploration of Large Language Model Society. NeurIPS 2023
10. Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023
11. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020
12. OpenAI Help Center: Projects in ChatGPT; Creating a GPT. help.openai.com
13. Anthropic Help Center: What are Projects? support.claude.com
14. Microsoft Learn: Copilot agents and Copilot Notebooks. learn.microsoft.com
15. Google: Gems in Gemini. support.google.com/gemini
16. IEEE CS Region 8 AI Caravan (2026). AI Administrator Track Curriculum. aicaravan.org

Figure: Alammar, J. and Grootendorst, M. (2024). Hands-On Large Language Models. O'Reilly Media. Figures also appear in the course tutorial albanna-tutorials.com/llm_agents.html.
